Syntax Is Not Function:
A Failure-Locus Audit of Photonic Netlist Generation

Dylan Yan
Shanghai Jiao Tong University
anzy200299@gmail.com

Abstract

A photonic netlist can execute without implementing the requested circuit. We audit five models on PICBench's 24 designs with five samples per design, without a restriction list or simulator feedback. We distinguish extraction failure, JSON parse failure, SAX/schema rejection, spectral mismatch, and spectral match under the benchmark's 1,000-wavelength, ten-decimal comparison of $|S|^2$. Qwen3.5-9B produces 4 simulable netlists and 1 match in 120 samples; 79 replies fail extraction or parsing. Four API models produce 64–98 simulable netlists but only 36–47 matches. Using failure categories observed on the test set, we generate 1,000 circuit-first questions and retain 985 spectrally checked teacher answers, including 84 model-dictionary repairs. At the loss-selected epoch-8 endpoint, teacher-answer fine-tuning yields 27 simulable netlists and 4 matches, compared with 31 and 0 under reference-JSON supervision. The advantage meets a pre-specified design-coverage threshold but falls below the sample-accuracy threshold. Earlier teacher-answer checkpoints yield 7 and 9 matches, showing sensitivity to checkpoint choice. Both epoch-8 students score below the base model on NetlistBench's structural SPICE evaluation. The results show that improved executability can coexist with limited spectral accuracy, checkpoint sensitivity, and no observed transfer to SPICE.

1 Introduction

PICBench asks a model to write, from natural language, a photonic netlist that names devices, binds their ports, and sets the parameters a circuit simulator accepts (Wu et al., 2025). Each of its 24 problems ships an expert netlist and a judge built on the SAX simulator (Laporte, 2023). A sample passes syntax when SAX returns a transmission, and functionally when that transmission matches the expert spectrum after rounding to ten decimals; prose that merely asserts an S-parameter is ignored. Figure 1 separates that JSON from the circuit it names and from a later chip layout. Figure 3 assigns every miss to the stage that rejected it.

Aggregate syntax and functional rates leave the causes of failure unresolved. A reply may fail before simulation because it cannot be parsed, or after parsing because its models or ports are invalid. Even a simulable circuit may produce the wrong spectrum. Distinguishing these outcomes is necessary to tell whether fine-tuning improves circuit behavior or only executability.

PhIDO, PICopilot, and Chat-to-Chip sit next to this benchmark and score a different object: GDS validity, design scripts, and metasurface geometry (Sharma et al., 2025; Jiang et al., 2026; Zhang et al., 2025). On PhIDO queries of at most fifteen components, the strongest models reach end-to-end $\mathrm{Pass@5}$ around 57%. A valid mask is a different outcome from a matching $|S|^2$. PICBench itself reports higher rates once a restriction list is added, or once the simulator error is returned for up to three revisions. We use the column with neither, Without EF. There the published functional $\mathrm{Pass@1}$ reaches 8.33% (Gemini 1.5 Pro) and functional $\mathrm{Pass@5}$ reaches 16.67% (o1-mini) (Wu et al., 2025).

VerilogEval, RTLLM, and AnalogCoder already score electronic netlists by compilation or by tests, with feedback in the AnalogCoder loop (Liu et al., 2023b; Lu et al., 2024; Lai et al., 2024). Pirouzan (2026) show that schema, topology, executability, and component-set agreement come apart: 16 of 150 circuits failed a topological check and still ran in ngspice, one of them a dangling resistor at 5.00 V instead of 2.50 V. NetlistBench applies a structural IR oracle to 2,342 SPICE cases (Ma et al., 2026). We extend this distinction to photonic netlist generation by auditing failures at each evaluation stage and testing supervision built from those failures. We compare answer distillation (Hinton et al., 2015) with direct supervision on the reference JSON, which already specifies the devices, wiring, and parameters. This control tests whether teacher replies provide a measurable benefit beyond the reference circuit itself.

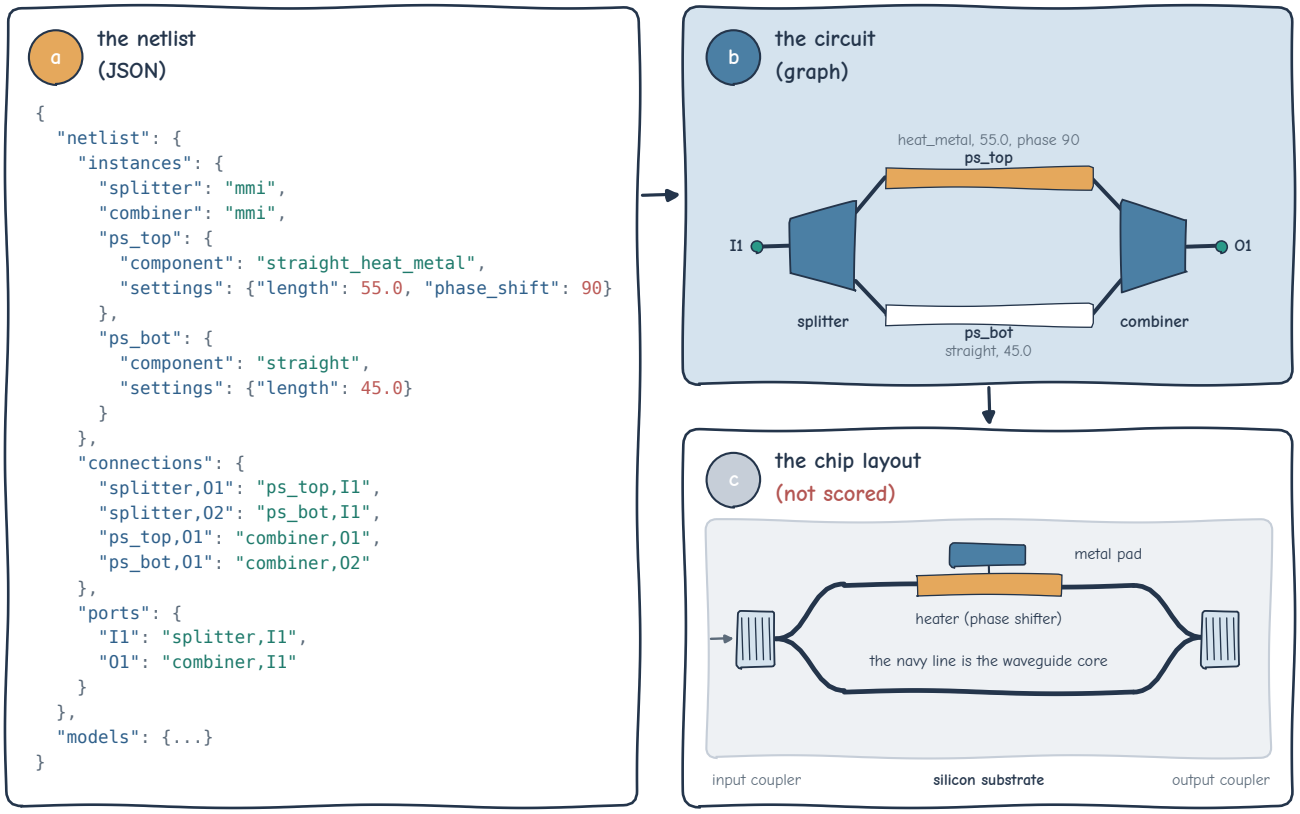


Figure 1: The object PICBench scores is the JSON netlist, not the mask. Panel (a) is one Mach–Zehnder: device instances, port bindings, a phase setting, and an arm length. Panel (b) is the circuit that JSON describes. Panel (c) is a schematic silicon-photonic layout of the same topology. A spectrum match does not place or route that layout, and it does not check a PDK cell. The drawing is an illustration. It is not one of the 24 expert files, and it was not passed through SAX.

We evaluate Qwen3.5-9B (Qwen Team, 2026), DeepSeek-V4-Pro (DeepSeek-AI, 2026), Qwen3.6-27B, GLM-5.3, and DeepSeek-V4.1-Flash on all 24 designs, with five samples each. The prompt, temperature, and judge are shared; model size, deployment, and reasoning configuration differ. We then compare two forms of supervision on the same synthetic questions: filtered teacher replies and reference JSON. Both improve executability, but their epoch-8 functional counts remain 4/120 and 0/120.

Per-sample accuracy and design-level coverage diverge: GLM-5.3 posts the highest functional Pass@1 (39.17%) yet a functional Pass@5 (54.17%) below Qwen3.6-27B's 70.83%, whose per-sample rate is lower but whose passing samples spread across more designs (Figure 2).

Our contributions are threefold:

- A stage-level audit of five models under Without EF, separating extraction and parsing failures, simulator rejection, and spectral mismatch. Among the four API models, only 48.0–56.3% of simulable netlists match the reference spectrum.
- A circuit-first generator of 1,000 questions targeting observed failure categories, with lexical and structural overlap checks. The categories are informed by test failures, and their manually assigned weights have not been compared with random circuit synthesis.
- A matched comparison of teacher-answer and reference-JSON supervision on 985 spectrally checked questions. At epoch 8, teacher answers improve functional design coverage over reference JSON but fall below the pre-specified sample-accuracy threshold. Both students perform worse than the base model on NetlistBench, providing no evidence of transfer to SPICE.

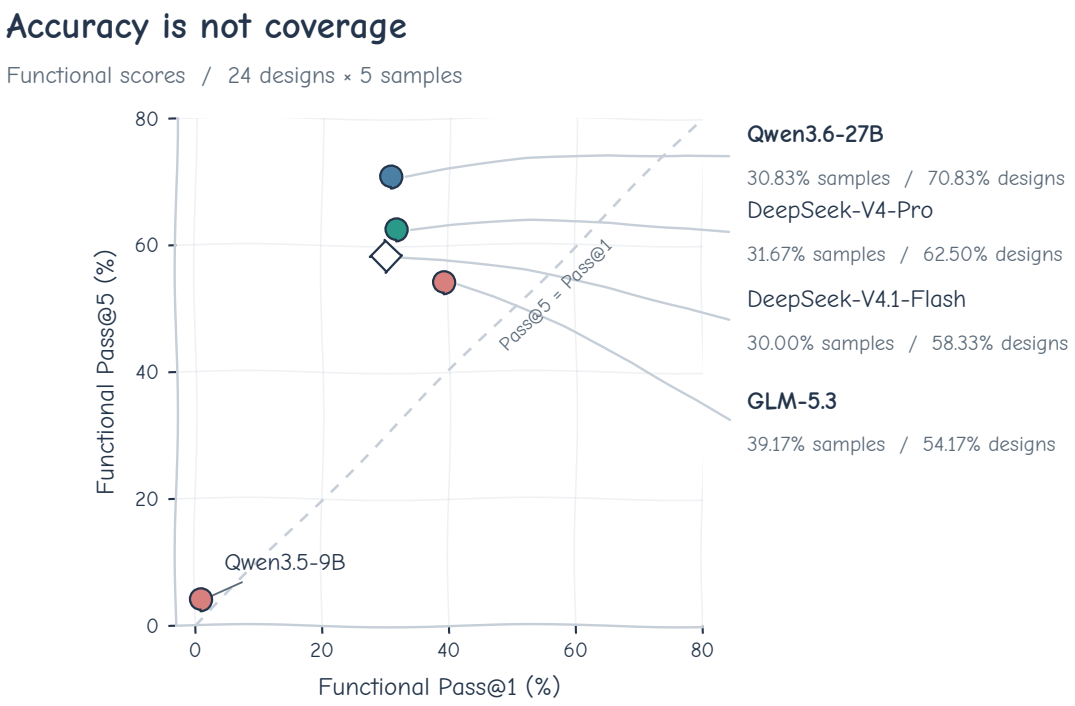


Figure 2: Functional accuracy and design coverage for the five measured models. Pass@1 is the fraction of matching samples; Pass@5 is the fraction of designs with at least one match. GLM-5.3 has the highest sample accuracy but covers fewer designs than Qwen3.6-27B. The axes use equal scales; the dashed line marks equal rates. Model size, deployment, and reasoning configuration differ across runs.

2 Related work

Photonic language-model generators. PICBench is a natural-language-to-netlist benchmark for photonic integrated circuits (Wu et al., 2025). Its 24 problems cover six optical-computing circuits, seven optical-interconnect circuits, nine optical switches, and two fundamental devices that themselves contain connections. Each problem ships a description, an expert netlist, and the SAX spectrum of that netlist. Syntax is "SAX can simulate." Function is "the simulated transmission matches the expert file." The original evaluation uses the unbiased Pass@k estimator of Chen et al. (2021) with $n = 5$ samples per design. We use that estimator and that judge without changing the rounding or the wavelength grid. Without restrictions and without feedback, the published syntax Pass@1 ranges from 8.33% to 16.67% and functional Pass@1 from 1.67% to 8.33%; adding the authors' restriction list, or returning the simulator error for up to three revisions, raises both numbers. We use the no-restriction, zero-feedback column, labeled Without EF, transcribed in Table 1, and we did not rerun the five commercial models.

PhIDO converts a natural-language photonic request into a GDS layout through a multi-agent loop (Sharma et al., 2025). Its testbench has 102 descriptions, from single devices to 112-component circuits, and its headline number is structural: whether the generated mask is a valid layout, not whether a compact-model spectrum matches an expert file. For queries of at most fifteen components, o1, Gemini-2.5-Pro, and Claude Opus 4 reach end-to-end Pass@5 around 57%. PICopilot generates design scripts rather than netlists, with retrieval and feedback, and reports that all 48 of its scripting tasks complete (Jiang et al., 2026). Chat-to-Chip fine-tunes language models to predict spectra and invert arbitrarily shaped metasurfaces (Zhang et al., 2025). The output is a geometry token sequence, not a SAX JSON graph, and none of these systems partitions load failures on a nested $|S|^2$ judge or separates a reference-JSON student from a teacher-answer student.

SAX is an S-parameter circuit simulator built on JAX (Laporte, 2023). Given a JSON netlist and a dictionary of device models, it returns the scattering matrix on a wavelength grid. PICBench builds models for waveguides, couplers, multimode interferometers, Mach–Zehnder stages, microrings, and phase shifters, and it simulates 1,000 wavelengths from 1.51 to 1.59 μm. A functional match is a property of those curves, inside a fixed compact model,

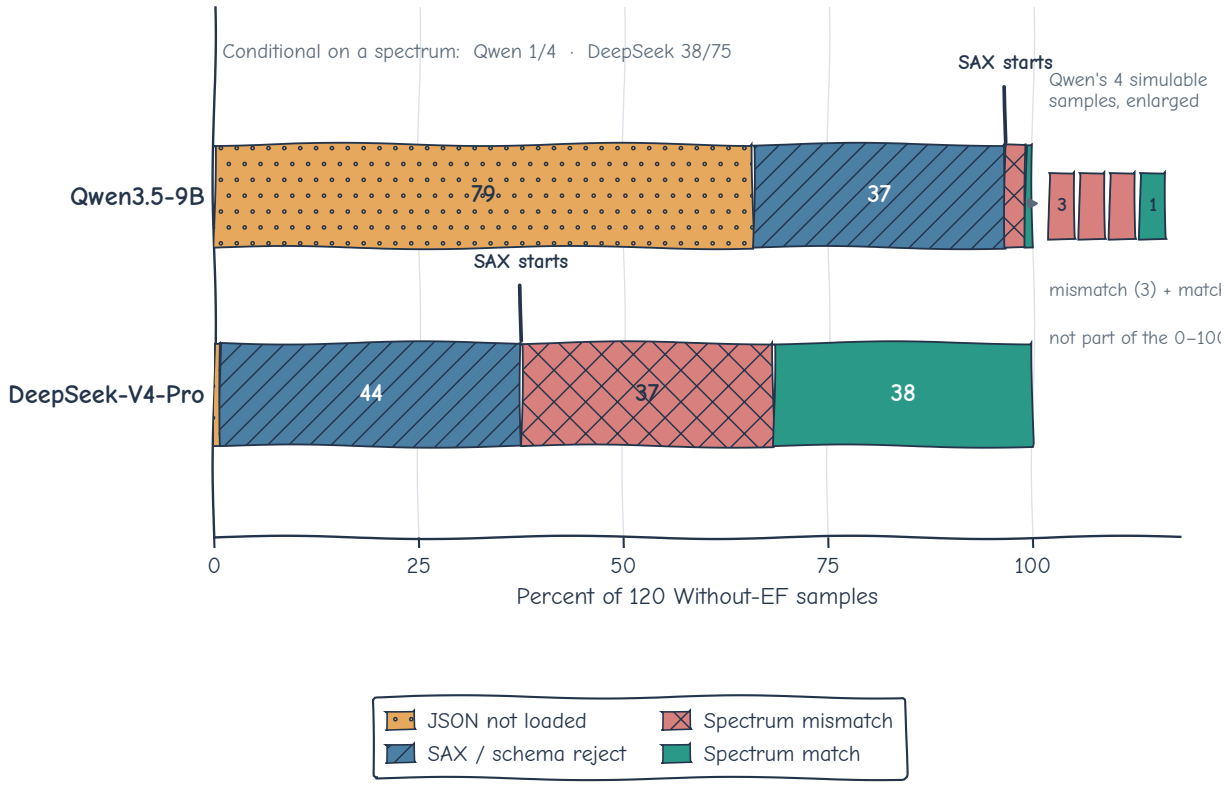


Figure 3: Each bar is 120 Without-EF samples, stacked by the mutually exclusive stage in which the judge rejected or accepted the netlist. "JSON not loaded" joins a missing result span (7 for Qwen3.5-9B, 1 for DeepSeek-V4-Pro) to a `json.loads` rejection (72 and 0). The navy tick marks where SAX starts: simulable netlists begin to its right. For Qwen that region is a thin sliver—three spectrum mismatches and one match—so the four cells on the right repeat those samples at equal width, marked enlarged, outside the 0–100 axis. Syntax pass is the mismatch plus match segments, and functional pass the match segment alone; counts are Table 6.

with layout, full-wave electromagnetics, and fabrication outside the judge.

Syntax versus function in electronic netlists. The electronic relatives of the task already treat compilation and tests as different events. VerilogEval and RTLLM score generated RTL by execution rather than by text overlap (Liu et al., 2023b; Lu et al., 2024). AnalogCoder generates PySpice, repairs against simulator feedback, and reports 20 of 24 analog tasks solved (Lai et al., 2024). ChipNeMo domain-adapts LLMs for an engineering chatbot, EDA scripts, and bug summaries; it is an adaptation result, not a nested syntax/function partition of a netlist benchmark (Liu et al., 2023a). Pirouzan (2026) makes the partition explicit for LLM-generated circuits: schema validity, topological validity, backend executability, and component-set agreement are four levels, and they are not nested. On 150 circuits, 16 were rejected by a topological validator and still executed in ngspice without a warning; a dangling resistor in a divider produced 5.00 V instead of 2.50 V while the simulator stayed silent. That is the electronic form of a legal-but-wrong netlist. NetlistBench asks a related question about SPICE rather than photonic JSON (Ma et al., 2026). Its oracle compares a canonical intermediate representation over 2,342 recognition and edit cases; a SPICE pass rate is not a PICBench Pass@k. Section 9 reports the 9B zero-shot score and both epoch-8 students on that oracle in separate tables, and the 2,342 released cases stay out of both training pools.

Answer distillation. We fine-tune a student on a stronger model's visible replies, a form of answer distillation (Hinton et al., 2015). The target includes the visible analysis and result block; the separate API reasoning trace is excluded from the loss. Our reference-JSON control uses the same questions and training procedure but directly supervises the reference circuit. This comparison isolates the effect of the supervision target within the chosen training procedure, rather than comparing circuit supervision with formatting alone.

3 Measurement protocol

3.1 Judge

For each decoded netlist, PICBench's `evaluate` returns a verdict string. We retain its official syntax and functional metrics and resolve the evaluation into five mutually exclusive outcomes: extraction failure, JSON parse failure, SAX/schema rejection, simulation-valid spectral mismatch, and spectral match.

Extraction fails when the saved reply has no usable `<result>` span, so nothing is passed to the judge. JSON parsing fails when `json.loads` rejects that span. A fence, a `//` comment, an ellipsis, or trailing prose can all land there, and none of them is a SAX result. Simulation-valid means SAX returns a finite spectrum on

$$\lambda_i = 1.51 + (1.59 - 1.51)\frac{i}{999}, \qquad i = 0, \ldots, 999,$$

in microns, with $|S_{pq}(\lambda_i)|^2$ recorded for every external input–output pair the candidate declares, and with the same number of curves as the expert file. PICBench calls every such reply a syntax pass, including replies whose spectrum does not match. A spectrum match requires those curves, rounded to ten decimal places, to be equal as sets; PICBench calls that a functional pass, and the rounding and set comparison are the benchmark's. A matching spectrum is a subset of the simulation-valid replies, so the two official rates share samples.

A parsed JSON object can still miss a model name, and a simulation-valid netlist can still disagree with the expert file.

3.2 Pass@k, and a conditional rate

Each design is sampled $n = 5$ times. If c of those samples pass a given criterion,

$$\mathrm{Pass@}k = \mathbb{E}_d\left[1 - \frac{\binom{n-c_d}{k}}{\binom{n}{k}}\right], \tag{1}$$

where the binomial ratio is replaced by 0 when $n - c_d < k$, the usual unbiased estimator (Chen et al., 2021). We report $k = 1$ and $k = 5$, averaged over the 24 designs that all have five completed samples. With $n = k = 5$, Pass@5 equals the fraction of designs with at least one passing sample, and with equal n, Pass@1 equals the number of passing samples divided by 120. One design moving from $0/5$ to $1/5$ changes Pass@1 by $1/120 = 0.83$ points and Pass@5 by $1/24 = 4.17$ points. Sample accuracy and design coverage therefore require separate thresholds: ten additional matching samples increase Pass@1 by $10/120$, whereas two newly covered designs increase Pass@5 by $2/24$, even if each contributes only one match. Section 8 specifies a threshold for each metric. The 120 draws are not treated as independent: five samples share a prompt, and the expectation in equation 1 is over designs.

We also report

$$\mathbb{P}(\text{function} \mid \text{syntax}) = \frac{\#\{\text{matching spectra}\}}{\#\{\text{simulable netlists}\}}.$$

This conditional rate measures spectral correctness among simulation-valid netlists. It complements Pass@k: increasing executability alone need not increase the number of functional matches and may lower the conditional match rate.

3.3 Prompt and decoding

The prompt is the one in PICBench's generator for a run with restrictions turned off and with zero feedback iterations. The system message is `system_prompt.txt`: a SAX JSON schema, the built-in device list, and the requirement that the reply contain an analysis followed by a `<result>` block. The restrictions file is not concatenated. One in-context example, the published Mach–Zehnder question and its published answer, precedes the test item. The user message is the problem text plus the published instruction, which asks for a JSON netlist and, in the same sentence, for advice "based on the restrictions." That sentence survives even though the restriction list is absent, because it is what `gen_data.py` sends, and the official client places the whole user message twice, once inside the seeded dialogue and once as the next turn; our calls do the same.

The published one-shot answer connects both waveguides to the same combiner input, which is illegal under the simulator's one-connection-per-port rule. We leave it in place because it is part of the prompt the Without-EF column used; the example does not make a legal netlist unreachable.

Qwen3.5-9B is the local post-trained checkpoint, loaded in bfloat16. Thinking is disabled, so the sampled tokens are the visible reply. Generation uses temperature 1, top-p 1, a torch seed equal to the sample index, a limit of 32,768 new tokens, and a stop string at `</result>`. DeepSeek-V4-Pro is the API model name `deepseek-v4-pro`, called on 20 October 2026 at temperature 1 with a 32,768-token completion budget and no stop string. The API returned a reasoning trace on every call. Scoring uses the netlist parsed from the string `reply_text` stores. That string is the visible `content` when `content` contains `<result>`, which is true of 118 of the 120 calls. On the other two calls the visible content has no such tag. Crossbar 4×4 sample 1 has the tag only in `reasoning_content`, and the scorer falls back to that trace. The stored netlist is rejected for the model name `osu`; it is neither simulation-valid nor a spectrum match, so the fallback leaves both Pass@k unchanged. It does move the stage table: under a visible-content-only rule this sample would be a second extraction failure and the model-name rejections would be 7 rather than 8. U-matrix sample 2 has the tag in neither field. That is the one extraction failure in Table 2 and Table 6. Ten reasoning traces contain `<result>`, but nine of those calls also carry the tag in the visible content, and the visible content is what was scored; the local 9B run has no separate reasoning field, and its judged string is the visible generation.

Both runs cover $24 \times 5 = 120$ samples. Every DeepSeek call ended with finish reason `stop`; one Qwen call hit the 32,768-token cap. The shared prompt, temperature, sample count, and judge do not make the runs a controlled comparison of reasoning. Qwen is a local 9B checkpoint with thinking disabled. DeepSeek-V4-Pro is an API model that returned a reasoning trace on every call, and we do not have its weight hash or a matched token budget for that trace.

3.4 Shared protocol

Relative to Table III Without EF, we share the problem texts, the system prompt, the in-context example, the double user turn, the absence of `restrictions.txt`, zero error feedback, $n = 5$, and the SAX judge. Our temperature is 1 and top-p is 1 for the local 9B run, the historical default of the chat interface the original code calls. The API request for DeepSeek-V4-Pro sets temperature 1 and does not set a separate top-p. The original paper does not publish a temperature, a token cap, or a seed, and we did not rerun its five commercial models, so the first five rows of Table 1 are transcribed from that paper and the last five are our measurements. Qwen3.6-27B, GLM-5.3, and DeepSeek-V4.1-Flash were evaluated subsequently on the same 24 problems, with the same prompt, temperature 1, sample count, and SAX judge. Qwen3.6-27B used a 65,536-token completion cap. GLM-5.3 used 131,072 tokens for 117 calls and 32,768 for three; DeepSeek-V4.1-Flash used 32,768 tokens for 119 calls and 131,072 for one. The five measured rows still do not hold scale, deployment, or the token budget fixed. Designs that are short of five samples stay out of every Pass@k in this paper.

4 Zero-shot results

Table 1: Syntax and functional Pass@k on PICBench, in percent. The first five rows are transcribed from Table III of Wu et al. (2025), Without EF, and were not rerun. The last five rows use the protocol in Section 3. Syntax includes functional matches.

Model	Pass@1		Pass@5	
	Syn.	Func.	Syn.	Func.
GPT-4	16.67	6.67	41.67	12.50
o1-mini	8.33	4.17	29.17	16.67
GPT-4o	14.17	4.17	37.50	4.17
Claude 3.5 Sonnet	13.33	1.67	20.83	8.33
Gemini 1.5 Pro	9.17	8.33	16.67	12.50
Qwen3.5-9B	3.33	0.83	16.67	4.17
DeepSeek-V4-Pro	62.50	31.67	95.83	62.50
Qwen3.6-27B	63.33	30.83	91.67	70.83
GLM-5.3	81.67	39.17	100.00	54.17
DeepSeek-V4.1-Flash	53.33	30.00	91.67	58.33

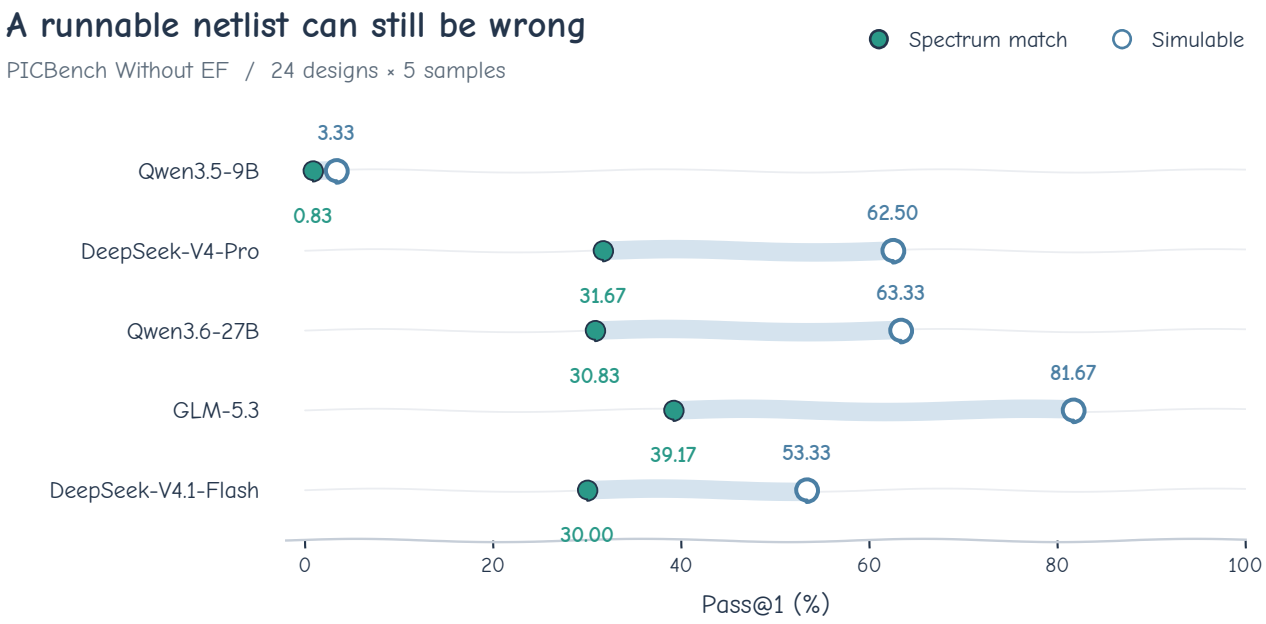


Figure 4: Syntax and functional Pass@1 for the five measured models. Open circles mark simulation-valid samples; filled circles mark spectrum matches. Each connecting segment is the rate of simulable but nonmatching samples. The four API models span 53.33–81.67% on syntax but only 30.00–39.17% on function. Model size, deployment, and reasoning configuration differ across runs.

Table 1 is the main measurement. Qwen3.5-9B produces 4 simulable netlists and 1 matching spectrum in 120 draws. Those four draws sit on four designs, one each, so the syntax Pass@5 is $4/24 = 16.67\%$ and the functional Pass@5 is $1/24 = 4.17\%$. The matching spectrum is one sample of the QPSK modulator. The other three simulable replies, on the direct modulator, the Mach–Zehnder modulator, and the nonlinear-sign gate, miss the expert spectrum. The remaining 20 designs are $0/5$ on both criteria. No optical switch is among the four simulable replies.

DeepSeek-V4-Pro produces 75 simulable netlists and 38 matching spectra. Syntax Pass@5 is $23/24$: the WDM demultiplexer is the only design with no simulable sample, and all five of its failures are port errors. Functional Pass@5 is $15/24$. The nine designs with no functional sample are the 8×8 Clements, Reck, crossbar, and Spanke–Benes meshes, the phase-shifter MZI, the nonlinear-sign gate, the QPSK modulator, and both WDM circuits. Direct modulation and the Mach–Zehnder modulator are $5/5$ functional. The phase-shifter MZI is $5/5$ syntax and $0/5$ functional: the model can emit a netlist the simulator accepts, and the spectrum is still wrong on every draw. Figure 4 shows the same split across all five measured models.

Table 2: Outcomes on the string the Without-EF judge actually scored. Extraction failure means that scored string had no usable result span. For DeepSeek, 118 visible contents contain the tag; one further netlist was read from the reasoning trace and then rejected for a model name, so the extraction-failure count stays 1. JSON parse failure means `json.loads` rejected that span. The next row is a parsed object that SAX or the model dictionary rejected. $\mathbb{P}(\text{function} \mid \text{syntax})$ is matching spectra over simulable netlists.

Quantity	Qwen3.5-9B	DeepSeek-V4-Pro
Simulable / 120	4	75
Matching / 120	1	38
$\mathbb{P}(\text{function} \mid \text{syntax})$	$1/4 = 25\%$	$38/75 = 50.7\%$
Extraction failure / 120	7	1
JSON parse failure / 120	72	0
Parsed, SAX or schema rejects / 120	37	44

The 9B model almost never reaches the simulator; DeepSeek-V4-Pro almost always does, and it is right on 38 of 75 simulable netlists. Moving mass from the unloadable rows into the simulable row would raise syntax Pass@1, not functional Pass@1, and Reference-JSON epoch 8 does exactly that: 31 simulable netlists against 4 at zero shot, with 0 spectrum matches.

The models succeed on different subsets of designs, so aggregate rates do not establish item-wise dominance. DeepSeek-V4-Pro's functional Pass@1 exceeds the transcribed Without-EF results, although differences in model deployment and undocumented historical decoding settings limit direct comparisons.

Table 3: Simulable / matching samples by PICBench family, out of the family total. Totals are 30 computing, 35 interconnect, 45 switch, and 10 device samples (six, seven, nine, and two designs, five each). DS-V4-Pro is DeepSeek-V4-Pro; V4.1-Flash is DeepSeek-V4.1-Flash.

Family	Qwen3.5-9B	DS-V4-Pro	Qwen3.6-27B	GLM-5.3	V4.1-Flash
Computing	1/0	15/6	19/3	27/1	13/3
Interconnect	2/1	19/8	25/14	21/9	15/13
Switch	0/0	31/19	26/17	41/33	29/15
Devices	1/0	10/5	6/3	9/4	7/5

Table 3 breaks the same 120 samples by PICBench family. Qwen's simulable replies are confined to interconnect and device problems plus the nonlinear-sign gate; no switch contributes. DeepSeek simulates every device sample and matches half of them, while GLM's functional mass concentrates on switches (33/45) and nearly vanishes on computing (1/30).

Table 4: DeepSeek-V4-Pro on the six architectures that PICBench poses at both 4×4 and 8×8. Each cell is the number of simulable samples and, after the slash, the number whose spectrum matches, out of five.

Architecture	4×4	8×8
Clements	4/2	2/0
Reck	2/1	2/0
Crossbar	3/1	3/0
Spanke	3/2	4/3
Benes	4/4	2/2
Spanke–Benes	4/4	3/0
All six	20/14	16/5

Width is informative for the model that can already emit meshes: across the six paired architectures, DeepSeek's functional count falls from 14/30 at width 4 to 5/30 at width 8, with Spanke the one exception. Qwen is $0/5$ on all twelve items. The drop is why the training generator in Section 6 never emits width 4 or 8, and per-design counts are in Appendix B.

4.1 Three further API runs

Qwen3.6-27B produces 76 simulable netlists and 37 matching spectra (syntax Pass@5 22/24, functional Pass@5 17/24, conditional match rate 48.7%). GLM-5.3 produces 98 simulable netlists and 47 matching spectra (syntax Pass@5 24/24, functional Pass@5 13/24, conditional match rate 48.0%), and every design has at least one simulable sample. GLM's functional Pass@1 is the highest of the five rows and its functional Pass@5 is below Qwen3.6-27B's, because more matching samples are concentrated on fewer designs (Figure 2). The phase-shifter MZI is again $5/5$ syntax and $0/5$ function for GLM, the same split DeepSeek shows on that item. Qwen3.6-27B matches the WDM demultiplexer on $4/5$ draws, a design DeepSeek-V4-Pro never simulates, and matches the U-matrix block on $0/5$, a design DeepSeek-V4-Pro matches on $3/5$. DeepSeek-V4.1-Flash produces 64 simulable netlists and 36 matching spectra (syntax Pass@5 22/24, functional Pass@5 14/24, conditional match rate 56.3%), and its failure stage differs from DeepSeek-V4-Pro's: 54 of 120 verdicts are port errors, against 24.

Table 5: Verdict strings for the three later API runs. The per-design `syntax` integer in the score files counts spectrum mismatches only, so this table is taken from the verdict strings. "Missing models" is the schema string inside an `other syntax` verdict, kept separate from the `wrong model` prefix.

Quantity	Qwen3.6-27B	GLM-5.3	Flash
Simulable / 120	76	98	64
Matching / 120	37	47	36
$\mathbb{P}(\text{function} \mid \text{syntax})$	$37/76$	$47/98$	$36/64$
Wrong-model prefix / 120	11	1	0
Wrong ports / 120	11	11	54
Missing models / 120	4	10	2
Other SAX or schema rejection / 120	18	0	0
Extraction or JSON failure / 120	0	0	0

Table 5 uses the official verdict strings, whereas Table 2 additionally re-audits extraction and parsing. No scored file in these three runs fails either step. Among simulation-valid samples, 39, 51, and 28, respectively, fail spectral comparison.

5 Where the 9B replies fail

The 120 Qwen verdicts separate replies rejected before simulation from legal netlists with wrong spectra. Table 6 labels the exact string passed to `evaluate`; it supersedes an earlier whole-reply heuristic that counted 97 surface-format failures.

Table 6: Stage of the stored netlist, the string the Without-EF judge scored. JSON parse reasons for Qwen, which can overlap inside one string, are ellipsis 44, trailing text 15, `//` 5, a fence that survived extraction 4, and other parse errors 4. "SAX or schema rejects" splits, for Qwen / DeepSeek, into SAX runtime 25/12, model name 9/8, and ports 3/24.

Stage of the stored netlist	Qwen3.5-9B	DeepSeek-V4-Pro
No usable result span	7	1
`json.loads` rejects the span	72	0
Parsed; SAX or the schema rejects it	37	44
Simulation-valid; spectrum differs	3	37
Spectrum matches the expert file	1	38
Total	120	120

Seventy-nine Qwen replies never yield an object the judge can load, 44 of the 72 parse failures because an ellipsis stands where repetitive entries should be. The length cap does not explain the table: the single capped reply is one sample of `WDM_mux`, already $0/5$ on the four uncapped samples.

The failures cluster on larger circuits: 45 of 60 Qwen samples on the twelve width-4 and width-8 meshes fail extraction or parsing; the same is true for 36 of 45 switch samples. The training pool does not test whether its shorter circuits address this limitation.

DeepSeek contributes one extraction failure and zero parse failures; of its 75 simulable netlists, 38 match the expert spectrum and 37 do not, so a simulation-valid netlist is still not a spectrum match. The dominant error that blocks simulation is the port map (24 replies), followed by other SAX errors (12) and undefined model names (8). On PICBench's two device designs, DeepSeek is $10/10$ simulation-valid but only $5/10$ functional. All five mismatches occur on the phase-shifter MZI.

5.1 Format repair

Before any fine-tune, a deterministic cleanup asks how much of the load failure is post-processing. The cleanup does not read the expert netlist and does not fill in an ellipsis; it strips fences, drops lines that are only `//` comments, and keeps the first JSON object. The cleaned strings are a modified protocol, kept out of Table 1.

On Qwen, the cleanup moves the 120 replies to 6 simulation-valid netlists and 2 spectrum matches (functional Pass@1 1.67%, Pass@5 8.33%). After cleanup, 40 originally unparseable replies reach SAX or schema validation but are rejected, and 17 still contain no complete object. Recovering a JSON object rarely recovers the intended circuit.

On DeepSeek, the same cleanup recovers one additional spectrum match; functional Pass@1 moves from 31.67% to 32.50%, Pass@5 stays $15/24$.

The distinction also follows directly from the stage counts: converting all 79 extraction or parse failures into simulable but incorrect netlists would raise Qwen's syntax count from 4 to 83 without adding a single functional match. This is a hypothetical bound on executability, not an estimate of what repair or training would achieve.

6 Failure-locus questions

We construct synthetic circuits separately from the 24 evaluation problems, using categories informed by the 9B model's test failures. We call this procedure *failure-locus verbalization* (Figure 5). Its six families—ellipsis, comment, wrong model, ports, function, and graft—target specific generation errors rather than PICBench's application categories. Graph construction precedes prose generation, and widths 4 and 8 are excluded. Table 7 summarizes the procedure.

Table 7: Failure-locus verbalization. The unit of sampling is a legal SAX graph. Generation does not read the 24 expert netlists. A later fingerprint screen does. The 9B text-heuristic labels choose the families; the weights are fixed and were not refit.

Move	What is sampled	What is left out
Index	9B text-heuristic labels	PICBench device families
Order	Graph $\rightarrow$ SAX $\rightarrow$ prose; JSON held out	Prose first; JSON in the prompt
Isolate	Contrast sets; width $\in \{2, 3, 5, 6\}$	Mix id with topology; width 4 or 8

The generator, with seed `20261020`, is the following procedure. The input is the PICBench device library and the earlier text-heuristic labels, not the re-audit in Table 6. The family weights were not refit after that audit. The output is a question whose prompt is prose and whose `reference` field is JSON.

1. Draw a family with the fixed weights in Table 8: ellipsis 280, comment 160, wrong model 280, ports 80, functional 120, graft 80.

2. Sample one free variable inside that family. Widths are uniform in $\{2, 3, 5, 6\}$. A wrong-model item copies the wiring of its contrast partner and changes one SAX identifier.

3. Reject the draw if a port is used twice, an external binding is dangling, the external width is 4 or 8, an eight-word shingle hits a test problem or the in-context example, or the text names 8-QAM, 64-QAM, or the test WDM wavelengths.

4. Call SAX on eight wavelengths in $[1.51, 1.59]$. Keep the draw only when every transmission is finite. The eight-wavelength check is separate from the 1,000-point equality test.

5. Write the prose from the graph, store the JSON as `reference`, and leave the restrictions file off.

No ablation against a random draw of the same size has been run, so the procedure does not say whether failure-locus sampling beats ordinary synthetic circuits; that comparison would need the same question count, student, budget, and Without-EF judge. The reference-JSON control in Section 8 is a different comparison—its epoch-8 score is 31 simulable netlists and 0 matches, and it is not a random-synthesis ablation.

The weights in Table 8 are a hand-set allocation, not fitted to the text-heuristic frequencies or to Table 6. Ellipsis and wrong-model each receive 280 items, matching the two skills the 9B run lacked: finishing a long repetitive object and keeping a SAX identifier with the wiring fixed. Comment items (160) force distinct numeric settings into a heater chain, while port items are only 80, although DeepSeek's dominant simulation block is the port map (24 of 120). No frequency-refit ablation has been run.

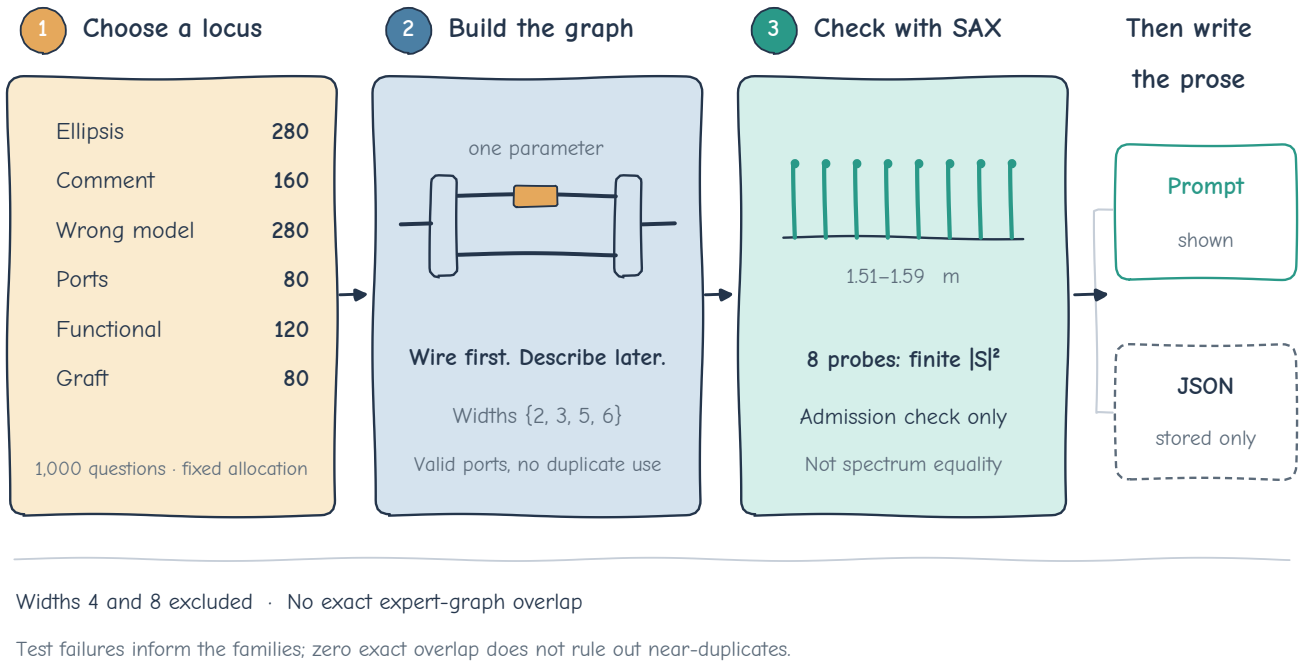


Figure 5: Circuit-first question generation. The six family counts are manually allocated from an earlier failure heuristic, not fitted to Table 6. A schematic circuit illustrates graph construction; eight wavelength probes check finiteness, not spectral equality. The prose enters the prompt; the reference JSON is stored separately. Widths 4 and 8 are excluded. No reference exactly matches an expert graph fingerprint, but near-duplicates and test-informed family selection remain possible.

The prompt also receives none of the format bans that PICBench derived from its own test-time errors; those bans are the restriction list, and using them here would leave the Without-EF protocol. The same 1,000 items are the question pool for both arms in Section 8, and copying `reference` supervises the gold circuit, including the schema.

6.1 One contrast set

Questions `q0441`, `q0548`, and `q0555` are one wrong-model group, `sand-17-81`. The records are in the question file shipped with the manuscript. Each prompt asks for the same path: a straight waveguide of length 17, a single-drive or interferometric core, and a heated tail with length 16 and phase 81. The three references differ only in the core identifier, `mzm`, `mzi`, or `mzm_dual`, and in the matching `models` entry; Appendix E prints the `q0441` prompt and its reference. The prompt names the SAX models in prose, and the JSON is not in the prompt, because a model that can see the reference can satisfy the item by copying it. The graph is built and checked before that prose is written, and the check that admitted the item is a finite $|S|^2$ on eight wavelengths; the teacher filter, when it is applied, uses the 1,000-point equality test against this reference. Those are different predicates.

6.2 From a label to a circuit

The generator draws a legal instance graph, checks that every external port binds to a real device port and that no port is used twice, and calls SAX. The draw is kept when every transmission on eight wavelengths in $[1.51, 1.59]$ is finite, and the prose description is then written from the same structure. The random seed is `20261020`, and all 1,000 references passed the finite-spectrum check.

The earlier heuristic categories guide the following circuit families; their allocation was fixed before the stage re-audit in Table 6:

- Ellipsis questions are parallel chains of identical blocks, width 2, 3, 5, or 6, so a correct JSON is long and repetitive; the test-mesh widths 4 and 8 never appear.
- Comment questions are chains of heaters in which every length and every phase differs, so the numbers have

to live in settings rather than in a copied block.

- Wrong-model questions are contrast sets that keep the wiring and swap the SAX identifier the 9B run confused, among `mzi`, `mzm`, `mzm_dual`, `straight`, `straight_heat_metal`, `coupler`, `mzi_2x2`, `mzi_ps`, and `OSU`; a student that copies a dialect without reading the identifier fails the contrast.
- Port questions state the external binding in the prompt, including a microring whose three outputs are all exported.
- Functional questions move one parameter away from the modulator and nonlinear-sign items that reached SAX in the 9B run, and they exclude the test parameter signatures.
- Graft questions wire two of those fragments in an order the 24 test items do not use: an interferometer into a microring, or one arm of a switch into a modulator.

Table 8 gives the counts, and template counts inside the loci are in Appendix D.

Table 8: The 1,000 training questions. Counts are from the generator manifest. None of these items is a PICBench test problem.

Locus	Count	What the prose specifies
Ellipsis	280	Parallel identical chains, width $\in \{2, 3, 5, 6\}$
Comment	160	Heater chain, distinct length and phase
Wrong model	280	Fixed wiring, SAX name is the variable
Wrong ports	80	External bindings, including every ring port
Functional	120	One-parameter move off a reached test item
Graft	80	Interferometer-then-ring, or switch-then-modulator
Total	1000	

These families target different requirements: parallel bundles require complete repetitive structures, while contrast sets require the correct SAX identifier even when port names are unchanged. Both supervision arms receive these same circuits; their comparison concerns the training target, not the choice of synthetic questions.

6.3 What the prompt already contains

Although reference JSON is hidden, every synthetic prompt names its SAX models and includes an explicit `instance,port` binding. Of the 1,000 prompts, 960 use the phrase "SAX model"; none of the 24 PICBench descriptions does. The teacher filter therefore measures transcription of an explicitly specified circuit, a narrower task than design from a functional specification. Its acceptance rate cannot be read as a PICBench score.

Table 9 compares the generated graphs with the 24 expert netlists. Ellipsis items are the long ones—the median bundle has 30 instances and 24 connections, the largest 60 and 54—while the other five families stay near three to seven instances. The expert files have a median of 7 instances and 8 connections and a maximum of 112 and 160, on the 8×8 meshes. Widths 2, 3, 5, and 6 are the widths the generator emits, and only the 280 ellipsis items carry a width; widths 4 and 8 are both the test mesh sizes and the scale at which DeepSeek's functional count already falls, in Table 4. Holding those widths out avoids copying a test mesh size, but it does not make the training graphs a disjoint circuit family: the 1,000 references collapse to 87 structural fingerprints, every fingerprint occurs at least twice, one occurs 73 times, and the settings signatures are far less repetitive, 968 of them distinct. Repeated topology with varying numbers is what the ellipsis and comment families are built to do, and it is not a broad sample of PICBench meshes; Section 6.4 records one component-multiset collision with a test expert.

Table 9: Size of the 1,000 generated references and the 24 PICBench expert netlists. Each cell is the median, with the maximum in parentheses. Only ellipsis items have a width, in $\{2, 3, 5, 6\}$. The two graft templates are balanced, 40 items each, so the graft row lists both sizes rather than their midpoint.

Set	n	Instances	Connections
Ellipsis	280	30 (60)	24 (54)
Comment	160	5 (7)	4 (6)
Wrong model	280	3 (3)	2 (2)
Ports	80	3 (3)	2 (2)
Functional	120	5 (7)	5 (6)
Graft	80	3 or 6	2 or 6
PICBench experts	24	7 (112)	8 (160)

6.4 Decontamination

Decontamination is lexical and structural: an eight-word shingle shared with any of the 24 problem texts, or with the in-context example, rejects the item. After generation, the longest shared word sequence has length five and occurs on 40 questions, the generic phrase "one input and one output"; the other 960 share no five-word sequence, and no question contains an eight-word copy of a test problem. The strings for 8-QAM and 64-QAM are rejected, and so is the test WDM wavelength tuple; mesh widths 4 and 8 are rejected by the port check, and the generator's width histogram on the items that carry a width is $\{2 : 59,\ 3 : 76,\ 5 : 70,\ 6 : 75\}$. Neither the expert netlists nor teacher replies to the 24 test prompts are used as training targets. Generation does, however, use the 9B failure labels on those prompts to choose circuit families. PICBench therefore serves as a test-informed evaluation rather than an untouched holdout. The generator does not read expert JSON; a subsequent audit reads the 24 expert netlists solely to compare their fingerprints with the generated references. Each fingerprint is the multiset of SAX model ids, the multiset of directed port edges after instance names are replaced by (model, local index), and the external port roles. None of the 1,000 training references shares a fingerprint with any of the 24 expert files, so the kept graphs are not those files; that does not make the sampler independent of the test labels, and it does not rule out a near-duplicate. Twenty functional items, all of template `coupler_phase_line`, share the nonlinear-sign expert's SAX-model multiset—three `coupler` devices and one `straight_heat_metal`. Those items have five connections and the expert has four, so the edge fingerprint differs, and the other 980 items share no component multiset with any expert. Thus the audit finds no exact fingerprint match, but it does identify shared component sets and does not establish independence from the test set. The teacher filter subsequently evaluates all references on the full 1,000-point grid. All 1,000 produce finite spectra, with no reference-build errors recorded.

7 Teacher filter

The teacher rollout uses the same prompt as Section 3, with model name `glm-5.3`, temperature 0, one completion per question, and a completion budget of 131,072 tokens. A trace is eligible for supervised fine-tuning when the visible content contains `<result>` and the enclosed JSON parses as an object with both a `netlist` and a `models` field; that visible content is the training target, and the reasoning trace is stored but never the target.

Before training, each teacher netlist is compared with its generated reference using the evaluation transmission test: a 1,000-point grid, ten-decimal rounding, and set equality of $|S|^2$ curves. Replies that fail this comparison are excluded. The filter is the same judge as Table 1, pointed at the generated references rather than at the 24 expert files. Run on every reply, it keeps 901 matches as stored, 84 that build only after the missing built-in SAX name is added to `models` as an identity entry, and drops 15 mismatches (985 kept). No finished reply fails to parse, and "match after the name repair" means SAX rejected the stored file and the same netlist matched once the missing name was added; only these repaired objects are written as the training target. Table 10 breaks the count down by locus.

Table 10: Teacher filter by the locus of the generated question. Kept is the sum of the first two columns. Functional questions were drawn 120 times and graft questions 80 times; the other loci were kept in full.

Locus	As stored	Name repair	Dropped	Kept
Ellipsis	225	55	0	280
Comment	160	0	0	160
Wrong model	252	28	0	280
Ports	80	0	0	80
Functional	110	0	10	110
Graft	74	1	5	75
Total	901	84	15	985

The 84 repairs add missing built-in names to `models` while the graph, the settings, and the ports stay as the teacher wrote them. Seventy-six add only `straight_heat_metal`, seven add only `straight`, and one adds the five names `mmi1x2`, `straight`, `straight_heat_metal`, `coupler`, and `mrr`. In each case the addition is exactly the built-in name missing from `models`, and the repaired object, not the raw file, is written as the training target. The 90.1% raw match rate is a rate on these 1,000 questions, not a prediction of PICBench performance.

8 Distillation against reference supervision

Table 11 and Figure 6 compare the base model with two students on the original 24 problems. Both students use the same 985 filtered questions. Reference-JSON SFT targets the reference object enclosed in result tags; Teacher-Answer SFT targets the filtered visible reply, including its analysis and result block, but excludes the separate API reasoning trace. Both targets specify a complete circuit, so Reference-JSON is a circuit-supervision control. No format-only control was run. The decision thresholds below were specified before either trained checkpoint was evaluated on PICBench.

We specified two alternative criteria for a practically relevant advantage of teacher answers over reference JSON: at least $10/120$ additional functional matches (8.33 percentage points of Pass@1), or at least $2/24$ additional covered designs on functional Pass@5. These are effect-size thresholds, not significance tests. We report each criterion separately because sample accuracy and design coverage can differ. A paired resample draws the 24 designs with replacement, 10,000 times, seed 20261021, and gives both students the same index on each draw. The observed gaps are $4/120$ matches and $4/24$ designs; their 2.5–97.5 percentile intervals are $1/120$–$8/120$ and $1/24$–$8/24$, respectively. The file is `logs/picbench/paired_student_bootstrap.json`. The point estimate meets the design-coverage criterion but not the sample-accuracy criterion. The intervals are descriptive and leave substantial uncertainty about the magnitude of the coverage benefit. Section 10 reports corresponding bootstrap intervals for the zero-shot rates. Even a larger advantage would not establish acquisition of general photonic design skills: the training prompts explicitly specify their circuits, and the evaluation contains only 24 test-informed designs.

Teacher-Answer SFT was started from the local post-trained Qwen3.5-9B checkpoint on the 985 kept visible replies. The update is full-parameter, on four A100-80GB GPUs, ZeRO-3, maximum length 5,583, per-device batch size 1, gradient accumulation 2, and a constant learning rate of 5×10^{-6}. Each stage is one epoch of 123 steps; epoch 1 uses warmup ratio 0.05 and seed 42, and later epochs use warmup 0 and seed $41 + t$ at epoch t. The launcher keeps a stage when its last-40-step mean loss is at most 98% of the previous stage's and stops otherwise, up to eight epochs. The kept target is the visible reply, including the analysis and the `<result>` block, in the chat template that embeds the duplicated user turn. The Reference-JSON arm uses that same wrapper, the same 2% rule, and as its target the question's `reference` object inside `<result>`, with no teacher analysis. The longest row measures 4,479 tokens, so `max_length` stays 5,583. Its last-40-step means are 0.00577428, 0.00000025, and then 0.00000000 for the remaining six epochs; each new mean was still at most 98% of the previous mean, including the zeros, so the 2% rule did not stop the run and the epoch cap did. Epoch 8 is the checkpoint named in `sft/picbench_reference/BEST`, decoded once under the same local protocol; these 24 problems did not choose it, and the score file is `logs/picbench/qwen35-9b-reference-sft-epoch8/summary.json`. Syntax / functional Pass@1 are 25.83% / 0.00% and Pass@5 are 45.83% / 0.00%: 31 simulable netlists and

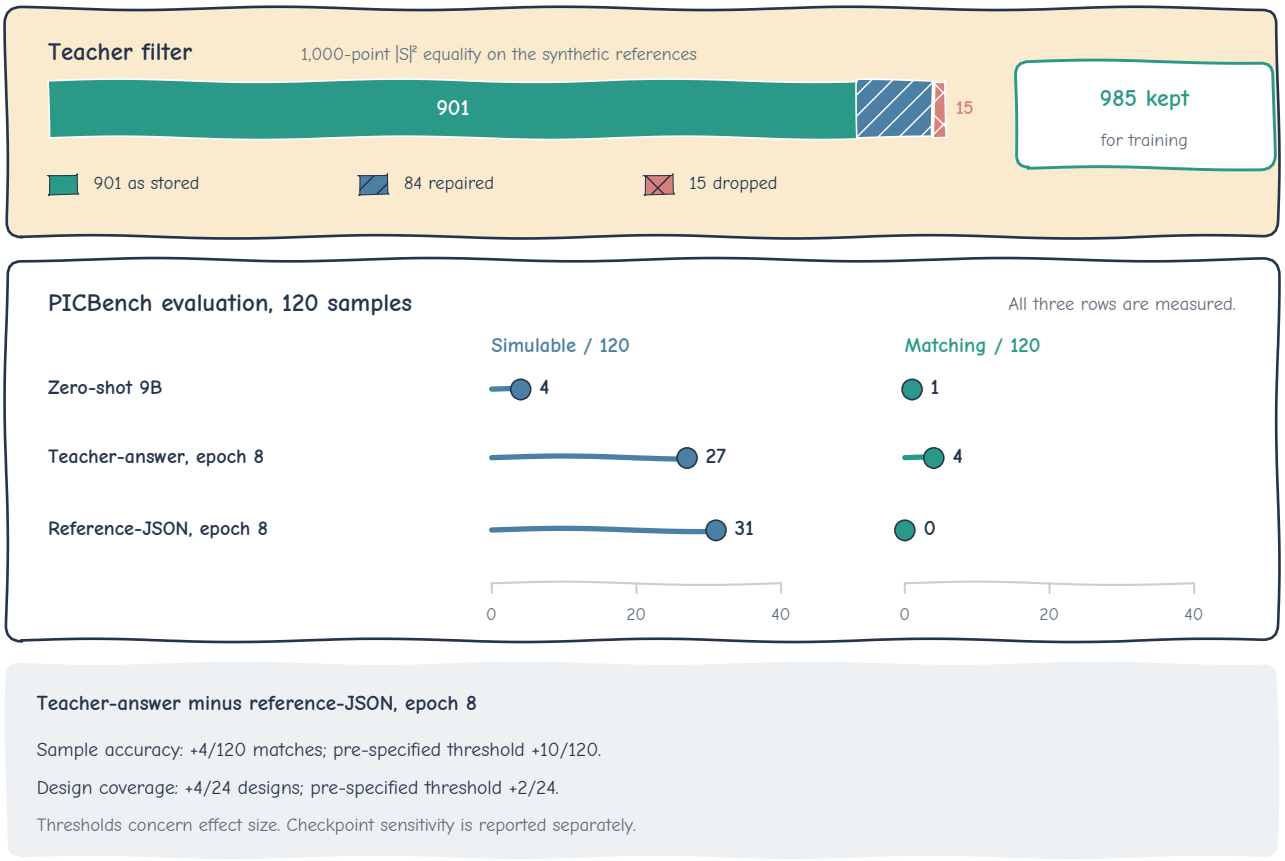


Figure 6: Teacher filtering and epoch-8 student evaluation. Of 1,000 teacher replies, 901 match the synthetic reference as stored, 84 match after a model-dictionary repair, and 15 are dropped. Both students train on the 985 retained questions. Points show counts out of 120 on a common 0–40 scale. Teacher-Answer yields fewer simulable samples than Reference-JSON (27 versus 31), but more spectral matches (4 versus 0). Its advantage meets the pre-specified design-coverage threshold and falls below the sample-accuracy threshold.

0 matches in 120 draws. Eleven of 24 designs have a simulable sample and none has a match. Of the 120 verdicts, 13 are wrong-model rejections, 32 port rejections, 1 a missing model, 43 other SAX or schema rejections, and 31 spectrum mismatches. Completions used 131 to 32,768 tokens, median 459, and 18 hit the cap; every saved file still contains a result span. Conditional on a simulable netlist the match rate is $0/31$, and family totals are computing $13/0$, interconnect $8/0$, switch $1/0$, and device $9/0$. The teacher-answer run also reached the eight-epoch cap: its last-40-step mean loss fell from 0.3055 to 0.0465 at every epoch, satisfying the 2% rule. Epoch 8 was scored once under the zero-shot protocol; the 24 test designs did not select the checkpoint. Syntax / functional $\mathrm{Pass@1}$ are 22.50% / 3.33% and $\mathrm{Pass@5}$ are 66.67% / 16.67%—27 simulable netlists and 4 matches in 120 draws, against 4 and 1 at zero shot. The four matches are one sample each of the 8×8 crossbar, the direct modulator, Spanke–Benes 4×4, and the WDM demultiplexer, and conditional on a simulable netlist the match rate is $4/27 = 14.8\%$.

Table 11: PICBench Without EF for the zero-shot 9B model and for both epoch-8 students. Syntax includes functional matches. Each epoch-8 row is the last checkpoint under the epoch cap. Neither was chosen by these 24 problems. Teacher-Answer exceeds Reference-JSON by $4/120$ functional matches, below the $10/120$ bar, and by $4/24$ designs, above the separate $2/24$ bar. The paired resample intervals are $1/120$ to $8/120$ and $1/24$ to $8/24$.

Condition	Pass@1		Pass@5		State
	Syn.	Func.	Syn.	Func.	
Zero-shot Qwen3.5-9B	3.33	0.83	16.67	4.17	measured
Reference-JSON, epoch 8	25.83	0.00	45.83	0.00	measured
Teacher-Answer, epoch 8	22.50	3.33	66.67	16.67	measured

A subsequent checkpoint audit evaluated Teacher-Answer epochs 1 and 2 under the same protocol. They yield 26 and 31 simulable samples, 7 and 9 matches, and 5 matched designs each (Appendix A). Thus the functional result is sensitive to checkpoint choice, and decreasing training loss does not imply improving PICBench performance. Epoch 8 remains the primary endpoint because the loss-based stopping rule selected it before PICBench evaluation; choosing epoch 2 retrospectively would use test performance for model selection. We report these post hoc results as a sensitivity analysis. Epochs 3–7 have not completed the same evaluation, so no complete functional learning curve is available. The thresholds were specified internally before evaluation; we do not claim a publicly timestamped preregistration.

9 The same 9B model on SPICE

NetlistBench is a different task and a different judge (Ma et al., 2026): the model must recognize or edit a SPICE netlist, and the oracle parses the reply and the reference into one intermediate representation, accepting the reply only when devices, terminals, parameters, and subcircuit interfaces match after numeric normalization. Symmetric passives compare terminals as sets. A pass here is not a SAX spectrum, and it is not a PICBench Pass@k; both models were decoded once per case on the official prompts, Qwen3.5-9B greedy with thinking off and a 4,096-token cap, DeepSeek-V4-Pro at temperature 0 with a 16,384-token cap.

Of the 1,542 manipulation cases, the 9B model passes 488 and DeepSeek 1,083; of the 100 equivalence judgments, 57 and 99; of the 700 recognition cases, 425 and 692. The family counts are in Table 12. On the 9B model every 9-step, 12-step, and 15-step flat sequence fails, as does every 9-step subcircuit edit and all 45 subcircuit expansions; DeepSeek passes all 45 expansions and 27 of 97 fifteen-step sequences. Device addition runs the other way: 61 for the 9B model, 53 for DeepSeek.

Table 12: NetlistBench, one completion each, official structural oracle. Equivalence is a yes/no label. Recognition is strict JSON. Neither pass count is a PICBench Pass@k.

Family	Qwen3.5-9B	DeepSeek-V4-Pro	n
Parameter edit	99	98	100
Device removal	96	99	100
Rename propagation	89	100	100
Device replacement	63	90	100
Device addition	61	53	100
Equivalence judgment	57	99	100
Connectivity edit	40	96	100
Flat compound, 3 steps	14	74	100
Subcircuit-internal, 3 steps	13	68	100
Subcircuit port swap	9	98	100
Flat compound, 6 steps	2	61	100
Subcircuit-internal, 6 steps	2	53	100
Flat compound, 9 steps	0	43	100
Subcircuit-internal, 9 steps	0	46	100
Flat compound, 12 steps	0	32	100
Flat compound, 15 steps	0	27	97
Subcircuit inline expansion	0	45	45
Recognition T1, device parameters	100	100	100
Recognition T2b, ordered terminals	100	99	100
Recognition T4, subcircuit ports	100	100	100
Recognition T5, instance port map	91	100	100
Recognition T3, node incidence	20	98	100
Recognition T2a, semantic pins	12	100	100
Recognition T6, terminal neighbors	2	95	100

The same protocol, greedy decoding with thinking off and a 4,096-token cap, was then applied to both epoch-8 checkpoints. Visible-reply epoch 8 passes 413 of 1,642 edits and 336 of 700 recognition items. Reference-JSON epoch 8 passes 407 of 1,642 and 195 of 700. The same 9B model at zero shot passes 545 of 1,642 and 425 of 700, the manipulation and equivalence rows of Table 12 plus its recognition rows. Both students sit below that zero-shot pair. Photonic visible-reply fine-tuning did not carry to SPICE. The family counts are in Table 13. On the reference student, subcircuit recognition is 0 of 200: every port-list reply fails to parse, and every instance-port reply has the wrong keys. Node incidence on that student is 0 of 100, all parse failures. The score files are `logs/netlistbench/qwen35-9b-visible-sft-epoch8-score.json` and `logs/netlistbench/qwen35-9b-reference-sft-epoch8-score.json`. These counts are not a PICBench Pass@k.

Table 13: NetlistBench, the two epoch-8 students, one completion each. The protocol and the official structural oracle are the ones in Table 12. These counts are not a PICBench Pass@k.

Family	Visible reply	Reference JSON	n
Parameter edit	96	97	100
Device removal	97	96	100
Rename propagation	60	55	100
Device replacement	31	31	100
Device addition	43	38	100
Equivalence judgment	50	54	100
Connectivity edit	20	28	100
Flat compound, 3 steps	5	2	100
Subcircuit-internal, 3 steps	6	3	100
Subcircuit port swap	4	3	100
Flat compound, 6 steps	0	0	100
Subcircuit-internal, 6 steps	1	0	100
Flat compound, 9 steps	0	0	100
Subcircuit-internal, 9 steps	0	0	100
Flat compound, 12 steps	0	0	100
Flat compound, 15 steps	0	0	97
Subcircuit inline expansion	0	0	45
Recognition T1, device parameters	100	100	100
Recognition T2b, ordered terminals	97	95	100
Recognition T4, subcircuit ports	96	0	100
Recognition T5, instance port map	35	0	100
Recognition T3, node incidence	7	0	100
Recognition T2a, semantic pins	0	0	100
Recognition T6, terminal neighbors	1	0	100

Appendix F lists the zero-shot first error codes. On that zero-shot 9B run, the split that matches the PICBench audit is recognition: the model emits legal JSON on 496 of 500 flat recognition replies and on all 200 hierarchical replies, yet T2a still fails 88 of 100 by `wrong_value` and T6 fails 90 of 100 by `wrong_keys`. Thus parseable JSON does not ensure correct terminal identification. DeepSeek's recognition replies are legal JSON on all 700 cases, with eight misses. That count is a structural IR result, not a PICBench spectrum match.

The same construction used for the photonic pool was pointed at these loci: a generator builds a legal SPICE netlist, applies an edit or a recognition question in code, and keeps the item only when the official parser and comparator accept the reference, which stays out of the prompt. Released step counts 3, 6, 9, 12, and 15, and subcircuit step counts 3, 6, and 9, are held out. An eight-word shingle against the 2,342 released instructions rejects an item, as does a rename-invariant fingerprint against the released netlists. The kept set has 1,000 items, seed 20261020, and zero eight-word overlaps. The allocation follows the measured fail counts by largest remainder: flat sequences 357, subcircuit-internal sequences 212, neighbor incidence 73, port swaps 68, semantic pins 65, node incidence 59, single-pin edits 45, inline expansion 33, equivalence 32, device addition 29, and device replacement 27. Families already near ceiling, including parameter edits and T1, receive no share. The 9B labels on the released cases choose those shares, so the pool is not independent of the test outcomes. The teacher rollout on these 1,000 questions finished with zero recorded failures, one file each. The official oracle accepts 970 of those visible replies: 354/357 flat sequences, 186/212 subcircuit sequences, 45/45 pin edits, 68/68 port swaps, 33/33 inline expansions, 29/29 additions, 27/27 replacements, 32/32 equivalence labels, 65/65 semantic-pin answers, 58/59 node-incidence answers, and 73/73 neighbor answers. Of the 30 misses, 25 do not parse (24 of them subcircuit sequences), two flat sequences mismatch a device, two subcircuit sequences mismatch a subcircuit, and one node-incidence answer has the right keys and the wrong value. No student has been trained on these replies, and the count is not mixed into the PICBench Pass@k table. The score file is `logs/netlistbench/glm53-teacher-1000-score.json`.

10 Limitations

The training procedure is test-informed diagnostic training: failure labels on PICBench determine the synthetic circuit families. Excluding exact references and screening overlap reduce direct copying but do not establish independent test-set generalization. A separate benchmark not used to design the generator is needed to assess that claim. The distillation comparison also uses one training run per arm; design-level bootstrap intervals do not capture variability across training seeds. Checkpoint sensitivity is substantial, and the available post hoc audit covers only epochs 1, 2, and 8.

The NetlistBench zero-shot numbers are one completion from each of two models, Table 12 reports those counts, and device addition runs the other way on that pair. Table 13 is the same oracle on the two epoch-8 students, also one completion each, and those counts stay out of the PICBench rates. The 1,000 SPICE questions are indexed by 9B failure counts. The official oracle accepts 970 GLM-5.3 teacher replies; no student has been trained on them. An IR match is neither a SPICE transient result nor a PICBench spectrum match.

The judge is a compact-model circuit simulator on a fixed C-band grid: two netlists that match to ten decimals can still differ geometrically, and a netlist that matches can still be unrealizable as a layout (Figure 1). We report the quantity PICBench defines.

The test has 24 designs and five samples, and the Qwen functional rate is one sample, so we report the counts alongside the rates. Samples within a design share a prompt, so an interval treating all 120 draws as independent ignores the design-level clustering. We therefore report both sample counts and design coverage and resample at the design level. A bootstrap over the 24 designs (10,000 resamples, seed 20261021) gives Qwen functional intervals of 0 to 2.50 (Pass@1) and 0 to 12.50 (Pass@5), and DeepSeek's 19.17 to 45.00 and 41.67 to 79.17. The same seed and the same design resample, paired across the two epoch-8 students, gives a functional-match gap of $4/120$ with interval $1/120$ to $8/120$ and a design-count gap of $4/24$ with interval $1/24$ to $8/24$. Each PICBench checkpoint was evaluated in one five-sample-per-design run; training and evaluation were not repeated across independent seeds.

Table 6 partitions the stored netlist, a reply can contain both an ellipsis and a comment, and the stage is the first reason `json.loads` fails. The format-repair numbers in Section 5.1 use a modified protocol. The synthetic prompts name the SAX models and the wiring, so the teacher filter is a transcription check on that pool.

DeepSeek-V4-Pro was queried by API model name on one date without an available weight hash; later deployments under the same name may differ. The historical commercial-model rows are transcribed from PICBench and were not rerun. The 9B run and the DeepSeek-V4-Pro run share the prompt, sample count, temperature, completion cap, and judge, but not the reasoning configuration or deployment. The three later API runs share the prompt and the judge, while Qwen3.6-27B and GLM-5.3 use larger completion caps than the 32,768-token cap of the base comparison.

The teacher filter covers all six loci. The 15 spectrum mismatches sit in the functional and graft questions, 10 and 5, so the other four loci were kept in full. That split is a count on this generator and leaves the decision rule in Table 11 unchanged.

11 Conclusion

The stage-level audit identifies two distinct obstacles in photonic netlist generation. Qwen3.5-9B usually fails before simulation, whereas the four API models frequently produce simulable circuits whose spectra do not match the reference. Executability alone therefore gives an incomplete account of performance under PICBench's compact-model judge.

Failure-informed synthetic supervision substantially increases the 9B model's executability, but functional gains remain small and sensitive to checkpoint choice. At the pre-specified epoch-8 endpoint, teacher-answer supervision exceeds reference JSON by four matching samples and four covered designs, meeting the coverage criterion but not the sample-accuracy criterion. Earlier checkpoints achieve more matches despite higher training loss. Because test failures informed the generator, these results support a diagnostic training study rather than an independent generalization claim. Both photonic students also perform worse than the base model on NetlistBench. Future comparisons need an untouched evaluation set, repeated training seeds, and a checkpoint-selection rule validated independently of the test tasks.

References

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

DeepSeek-AI. Deepseek-V4 preview: Entering the era of affordable million-token context. https://www.deepseek.com/en/news/v4-preview/, April 2026. Queried on 20 October 2026 under the API model name `deepseek-v4-pro`.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015. URL https://arxiv.org/abs/1503.02531.

Xiaohan Jiang, Zeyu Li, Wei Zhang, and Jiang Xu. PICopilot: An LLM-based agentic framework for assisting photonic integrated circuit design via script generation. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2026. arXiv:2608.01791.

Yao Lai, Sungyoung Lee, Guojin Chen, Souradip Poddar, Mengkang Hu, David Z. Pan, and Ping Luo. AnalogCoder: Analog circuit design via training-free code generation, 2024. URL https://arxiv.org/abs/2405.14918.

Floris Laporte. SAX: Circuit simulator for photonic integrated circuits. https://github.com/flaport/sax, 2023. Scoring in this paper uses `sax` 0.13.3 and `jax` 0.4.34.

Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, et al. ChipNeMo: Domain-adapted LLMs for chip design, 2023a. URL https://arxiv.org/abs/2311.00176.

Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023b.

Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024.

Jiarui Ma, Jianghan Wang, Yuheng Ma, Ziyi Zhuang, and Xiaoguang Liu. NetlistBench: Evaluating LLM reliability in SPICE netlist recognition and manipulation. In *ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, 2026. arXiv:2608.12197, https://github.com/WoshiMayou/NetlistBench.

Ali Hedayati Pirouzan. Validation and simulation catch different errors: Four levels of evaluation for LLM-generated circuits, 2026. URL https://arxiv.org/abs/2609.26830.

Qwen Team. Qwen3.5: Towards native multimodal agents. https://qwen.ai/blog?id=qwen3.5, February 2026. The student model is the post-trained `Qwen3.5-9B` checkpoint.

Ankita Sharma, Yuqi Fu, Vahid Ansari, Rishabh Iyer, Fiona Kuang, Kashish Mistry, Raisa Islam Aishy, Sara Ahmad, Joaquin Matres, Dirk R. Englund, and Joyce K. S. Poon. AI agents for photonic integrated circuit design automation, 2025. URL https://arxiv.org/abs/2508.14123. PhIDO: natural language to GDS; structural validity, not a performance-qualified spectrum.

Yuchao Wu, Xiaofei Yu, Hao Chen, Yang Luo, Yeyu Tong, and Yuzhe Ma. PICBench: Benchmarking LLMs for photonic integrated circuits design, 2025. URL https://arxiv.org/abs/2502.03159.

Huanshu Zhang, Lei Kang, Sawyer D. Campbell, and Douglas H. Werner. Chat to chip: Large language model based design of arbitrarily shaped metasurfaces. *Nanophotonics*, 2025. arXiv:2509.24196.

A Checkpoint sensitivity

Table 14 reports the last-40-step mean training loss for all eight epochs and all available Teacher-Answer PICBench evaluations. Losses are read from the saved stage logs. A dash denotes an unmeasured endpoint, not a zero. Reference-JSON losses from epoch 3 onward round to zero at the stored precision.

Table 14: Training loss and checkpoint sensitivity. PICBench counts are out of 120 samples or 24 designs. Epochs 1 and 2 were evaluated post hoc; epoch 8 is the primary endpoint.

Epoch	Teacher loss	Reference loss	Simulable	Matches	Designs
1	0.30548500	0.00577428	26	7	5
2	0.24988724	0.00000025	31	9	5
3	0.19529728	0.00000000	–	–	–
4	0.14603044	0.00000000	–	–	–
5	0.11129282	0.00000000	–	–	–
6	0.08268923	0.00000000	–	–	–
7	0.06188596	0.00000000	–	–	–
8	0.04648809	0.00000000	27	4	4

B Per-design counts

Table 15 lists every design, out of five samples. Each cell is simulable/matching (s/m); syntax counts SAX-returned spectra, functional is the matching subset. Qwen, at zero shot, is $0/0$ on twenty designs; its four exceptions are the direct modulator $(1/0)$, the Mach–Zehnder modulator $(1/0)$, the nonlinear-sign gate $(1/0)$, and the QPSK modulator $(1/1)$.

Table 15: Per-design counts, out of five. Each cell is simulable/matching (s/m). Epoch 8 is the Teacher-Answer checkpoint of Section 8, and Ref. 8 is the Reference-JSON checkpoint. Neither was chosen by these 24 problems. The three API columns are the verdict strings behind Table 5.

Design	Family	Qwen3.5	DS-V4	Qwen3.6	GLM-5.3	Flash	Epoch 8	Ref. 8
Clements 4×4	Computing	0/0	4/2	5/2	4/0	1/0	0/0	3/0
Clements 8×8	Computing	0/0	2/0	2/1	4/0	2/0	0/0	2/0
Reck 4×4	Computing	0/0	2/1	5/0	5/0	2/1	1/0	5/0
Reck 8×8	Computing	0/0	2/0	0/0	4/0	2/0	0/0	2/0
Nonlinear-sign gate	Computing	1/0	2/0	3/0	5/1	2/0	0/0	0/0
U-matrix block	Computing	0/0	3/3	4/0	5/0	4/2	2/0	1/0
Direct modulator	Interconnect	1/0	5/5	3/3	5/1	4/4	3/1	5/0
QPSK modulator	Interconnect	1/1	4/0	4/1	1/0	3/1	2/0	2/0
8-QAM modulator	Interconnect	0/0	3/1	5/2	3/0	5/5	1/0	0/0
64-QAM modulator	Interconnect	0/0	3/1	5/3	2/0	2/2	0/0	0/0
WDM multiplexer	Interconnect	0/0	3/0	3/0	2/0	0/0	2/0	1/0
WDM demultiplexer	Interconnect	0/0	0/0	4/4	4/4	1/1	2/1	0/0
Optical hybrid	Interconnect	0/0	1/1	1/1	4/4	0/0	1/0	0/0
Switch 2×2	Switch	0/0	5/3	4/3	5/1	4/0	3/0	1/0
Crossbar 4×4	Switch	0/0	3/1	4/2	5/5	3/0	2/0	0/0
Crossbar 8×8	Switch	0/0	3/0	3/1	5/5	4/0	1/1	0/0
Spanke 4×4	Switch	0/0	3/2	2/0	5/5	3/3	0/0	0/0
Spanke 8×8	Switch	0/0	4/3	0/0	3/3	5/5	0/0	0/0
Benes 4×4	Switch	0/0	4/4	4/4	4/4	1/1	2/0	0/0
Benes 8×8	Switch	0/0	2/2	4/4	5/5	3/3	0/0	0/0
Spanke–Benes 4×4	Switch	0/0	4/4	2/2	5/5	3/3	1/1	0/0
Spanke–Benes 8×8	Switch	0/0	3/0	3/1	4/0	3/0	1/0	0/0
Mach–Zehnder modulator	Device	1/0	5/5	3/2	4/4	2/2	1/0	4/0
MZI with phase shifter	Device	0/0	5/0	3/1	5/0	5/3	2/0	5/0

Qwen extraction or parse failures on the twelve meshes of width 4 or 8 sum to $45/60$, and summing the per-design counts recovers Table 3: DeepSeek-V4-Pro is $15 + 19 + 31 + 10 = 75$ simulable and $6 + 8 + 19 + 5 = 38$

functional, Qwen 4 and 1, and Qwen3.6-27B $19+25+26+6=76$ simulable and $3+14+17+3=37$ functional. Epoch 8 sums to 27 simulable and 4 functional, while Reference-JSON epoch 8 sums to $13+8+1+9=31$ simulable and 0 functional, the totals in Table 11.

C Prompt, decoding, and judge

The message list for the PICBench runs in this paper is:

1. system: PICBench `system_prompt.txt`, with no restriction list appended;
2. user: the published one-shot Mach–Zehnder question;
3. assistant: the published one-shot answer, which we do not edit;
4. user: the test problem, then the sentence that asks for a JSON netlist and for advice based on the restrictions;
5. user: the same text as the previous turn.

The fifth message is how `ASK_LLM_iterate` is written: the seeded dialogue already contains the question and the iterate call appends it again, a behavior we reproduce rather than "correct."

Local Qwen generation stops when the string `</result>` is emitted or at 32,768 new tokens, and the API calls have no stop string. Netlists are parsed by splitting on the result tag, dropping a closing tag, and stripping a surrounding markdown fence if one is present. The fence count in Table 6 is the primary label stored in the score file, not a second parse, and functional and syntax-only labels are assigned only after SAX runs.

The score files record 120 replies for each completed PICBench run in this paper, five samples on each of the 24 designs. Qwen completion lengths range from 723 to 32,768 tokens, median 8,170, with one cap hit, and DeepSeek completion lengths range from 954 to 27,844 tokens, median 5,420, with zero cap hits and finish reason `stop` on all 120 calls. Package pins for the judge are SAX 0.13.3 and JAX 0.4.34, the versions PICBench documents and the versions in the simulator environment used here.

D Question generator

The generator's rejection checks, applied before a question is kept, are:

- the instance graph uses only the device ports declared for its SAX models;
- no port is connected twice, and every exported binding points at a free port;
- the number of external inputs is not 4 or 8;
- SAX returns a finite spectrum on eight probe wavelengths;
- the question shares no eight-word shingle with the 24 problem statements or the in-context example;
- the question does not name 8-QAM or 64-QAM and does not contain the test WDM wavelength set.

The manifest records seed `20261020`, 1,000 kept questions, maximum word-overlap 5, overlap histogram $\{0:960,\ 5:40\}$, and a SAX acceptance count of 1,000. Template counts inside the loci are 280 parallel bundles, 160 heater chains, 120 one-port cores, 80 two-port cores, 80 single wires, 7 ring-port items, 73 splitters, 60 asymmetric interferometers, 40 modulator lines, 20 coupler-phase lines, 40 interferometer-into-ring grafts, and 40 switch-into-modulator grafts.

The reference netlist is the supervision for Reference-JSON SFT and the spectrum target for the teacher filter, and it is not a field the prompt renderer prints. A teacher trace is eligible only after the visible-content parse succeeds and the PICBench transmission test matches this reference; that second test has been run on all 1,000 teacher replies and is reported in Table 10.

E A worked wrong-model item

The prompt below is `q0441`, group `sand-17-81`, with its stored heading omitted. Questions `q0548` and `q0555` keep the wiring and settings but replace the core with `mzi` and `mzm_dual`. The prompt explicitly specifies the models, settings, and port bindings.

```
Build a one-input, one-output path of three instances: lead, core,
and tail, in that order.
lead is SAX model straight with length 17 microns. Map wg to straight.
core is a single-drive modulator. Its SAX model is mzm, at default
settings. Map mzm to mzm.
tail is SAX model straight_heat_metal with length 16 microns and
phase_shift 81 degrees. Map ps to straight_heat_metal.
Connect lead,O1 to core,I1 and core,O1 to tail,I1.
Bind circuit port I1 to lead,I1 and circuit port O1 to tail,O1.
```

The stored reference is

```
{
  "netlist": {
    "instances": {
      "lead": {"component": "wg", "settings": {"length": 17}},
      "core": "mzm",
      "tail": {"component": "ps",
               "settings": {"phase_shift": 81, "length": 16}}
    },
    "connections": {"lead,O1": "core,I1", "core,O1": "tail,I1"},
    "ports": {"I1": "lead,I1", "O1": "tail,O1"}
  },
  "models": {"wg": "straight", "mzm": "mzm",
             "ps": "straight_heat_metal"}
}
```

A long item has the same kind of prompt: `q0001` is an ellipsis bundle of width 6 and depth 7, 48 instances, and the prose lists every heater phase. A graft that omits the phrase "SAX model," `q0921`, still writes "split is mmi1x2," the lengths, the ring wavelength, and the connection list. None of these prompts is a PICBench-style functional specification.

F NetlistBench failure codes

Among the 9B model's 1,097 edit-root failures, the first error code is a device field mismatch in 328, a subcircuit-body mismatch in 278, an extra device in 233, a missing device in 151, a parse error in 63, a wrong equivalence label in 43, and an extra subcircuit in 1. On device addition, 26 of 39 failures change the node sequence and 13 keep that sequence and differ in the value field; two of the 26 also change the node set. The 13 value misses include unit strings such as `3pf` where the reference has `3p`, and the oracle does not treat those strings as equal. On compound sequences that emit a step hint, the first failed step is a pin reconnect in 217, a replacement in 189, an addition in 132, a rename in 80, a removal in 65, and a parameter edit in 18. DeepSeek's 460 edit-root failures are a device field mismatch in 322, a subcircuit-body mismatch in 133, a parse error in 2, a missing device in 2, and a wrong equivalence label in 1. Where a compound failure emits a step hint, the first failed step is an addition in 338, a replacement in 36, a parameter edit in 8, and a pin reconnect in 7.

G Reproducibility

The PICBench judge used here is SAX 0.13.3 and JAX 0.4.34. The local student checkpoint is the post-trained Qwen3.5-9B weights, in Hugging Face form, loaded in bfloat16 with thinking disabled; the model card identifies that directory with `Qwen/Qwen3.5-9B`, and no separate content hash was stored. DeepSeek-V4-Pro

was the API name `deepseek-v4-pro`, called on 20 October 2026 at temperature 1 with a 32,768-token completion budget and no stop string; Qwen3.6-27B was the API name `qwen3.6-27b`, called on 21 October 2026 with a 65,536-token completion budget; GLM-5.3 was the stored name `glm-5.3`, called on 20 October 2026, with 117 of those calls on a 131,072-token budget and three on 32,768. DeepSeek-V4.1-Flash was the API name `deepseek-v4.1-flash`, called on 21 October 2026, and 119 of its scored files use a 32,768-token budget while one uses 131,072, as recorded in Section 3. Section 4.1 records the verdict counts, and the visible-content fallback onto `reasoning_content` is the one in Section 3. Question generation used seed 20261020, the teacher was `glm-5.3` at temperature 0 with a 131,072-token budget, and the filter report is the file named in Section 7. Teacher-Answer SFT uses the hyperparameters in Section 8, and the epoch-8 PICBench decode is the score file named there. The Reference-JSON control uses the same hyperparameters, the same 2% stopping rule, and the question reference inside `<result>`; its epoch-8 score file is `logs/picbench/qwen35-9b-reference-sft-epoch8/summary.json`, with 31 simulable netlists and 0 matches. The NetlistBench scores for the two epoch-8 students are `logs/netlistbench/qwen35-9b-visible-sft-epoch8-score.json` and `logs/netlistbench/qwen35-9b-reference-sft-epoch8-score.json`. The 1,000 SPICE teacher replies are scored in `logs/netlistbench/glm53-teacher-1000-score.json`, 970 passes, and no student was trained on them. The project repository is https://github.com/anzy200299-ui/optics-agent. This release includes the project source code, evaluation scripts, configuration files, manuscript, and figure-generation scripts. The synthetic question sets, per-sample score files, teacher-filter report, and checkpoint loss logs are not included in this release. The repository therefore provides a partial code release rather than the complete experimental artifacts required for independent reproduction.